Elaboration of the project to develop a workflow visualizing ESP surfaces of a molecule

Methods

Felix Laufer

Frontiers in applied drug design

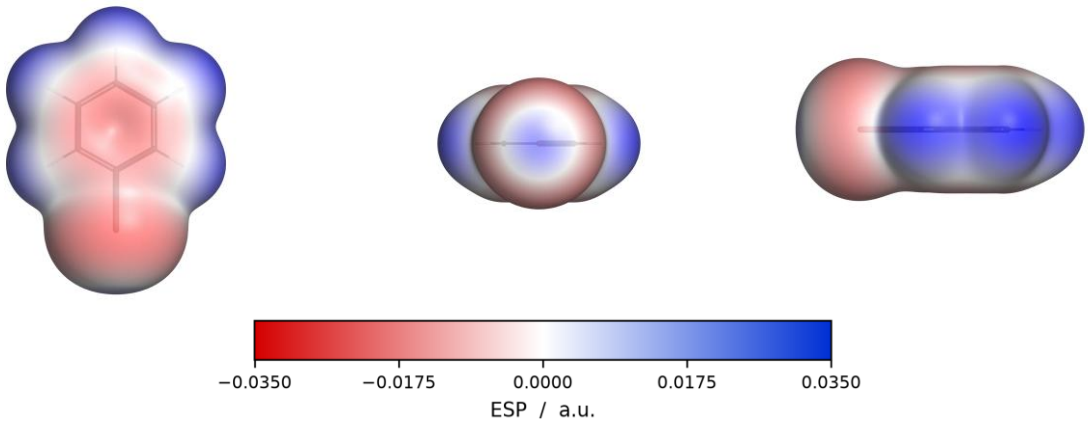

Figure 1: Visualization of Bromobenzene with the PyMol pipeline at ρ = 0.001 a.u, left is the view from the top, centre the axial presentation of the Halogen, and on the right side in-plane profile

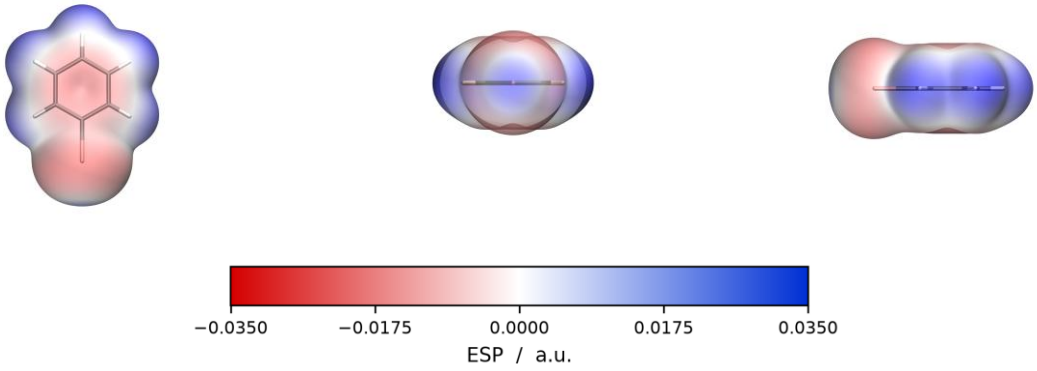

Figure 2: Visualization of Bromobenzene with the VMD pipeline at ρ = 0.001 a.u, left is the view from the top, centre the axial presentation of the Halogen, and on the right side in-plane profile

This document describes the implementation, data and results can be found in ProjectElaboration.pdf

1. PyMol pipeline

As stated in the *SoftwareEvaluation.pdf* document, for convenience reasons the project was started by implementing an ESP visualization pipeline within PyMol, it was already installed and I had a brief introduction via the introduction tasks. Together with Claude Cowork used as a software developer, the pipeline was implemented step by step and underwent heavy testing in the process.

As the provided electron density *td.xyz* and electrostatic potential *tp.xyz* files are not practicable for the visualization, they needed conversion into a shorter format, the Gaussian cube (*.cube)* format. That's the main purpose of the xyzToCube.py script.

1.1 xyzToCube.py

Within this script the conversion of the *turbomole pointval* grid files to the *.cube* format is conducted. The provided *tp* and *td.xyz* files carry the information about the electron density and the potential at each point in the grid, stored with the full coordinates of the grid.

The grid in the provided benzene (bromobenzene, iodobenzene and chlorobenzene) files ranges from -15, -15, -15 to 15,15,15 with a step size of 0.12 Bohr. For every grid point all three coordinates are explicitly stored, which makes up to 15.813.251 lines and a file size of 1.25GB. A cube file therefore only saves the important values (density and potential) with an implicit grid.

The grid in the provided pyridine derivate files ranges from -20, -20, -20 to 20, 20, 20 with a step size of 0.16 Bohr. Which also makes up to 15.813.251 lines and a file size of 1.25GB. Within the *.xyz* potential and density files x varies fastest, as stated in 1.1.3 within the .cube files z varies fastest, so the values had to be reordered in the process.

The conversion to Gaussian cube files, storing the grid implicitly saves of the coordinates and therefore shrinks the file size to 201.1 MB.

1.1.1 Converting units

Another point that this script resolves, is that the most structure files come in Ångstrom, therefore they need a conversion constant from ANGSTROM_PER_BOHR, which is outsourced to a second *constants.py* script, where they are imported from.

The workflow was implemented with all 3 common types of structural formats. *.mol*,*.sdf* and *.xyz* file formats are accepted and the pipeline can handle them. The provided .xyz structural files come in Ångstrom, so they need this conversion, if someone uses different structure files formatted in Bohr, they are not converted and left untouched. Line 201-250 in the script.

1.1.2 Reading the Turbomole grid file

From line 251 the reading of the Turbomole file starts, it essentially reads in the file as chunks of data (16MB at once, defined with the CHUNK_BYTES constant). It first parses the header to get the needed information for the generated .cube file header and then reads in the values of the density and the potential.

1.1.3 Writing the cube file

For security reasons, the file is named *.cube.part* until the *.cube* file is completely done. If a run crashes at the point of generating the cube file, it is indicated by the *.part* suffix, so no incomplete *.cube* is used for the subsequent pipeline. In the end the *.cube* file as stated stores the grid implicitly. The first 251 values therefore range from -20, -20, -20 to -20, -20, 20 for the pyridines and from -15, -15, -15 to -15, -15, 15 for the benzene files. The next 251 values than range from -20, -19.84, -20 to 20, -19.84, -20 for the pyridines and from -15, -14.88, -15 to 15, -14.88, -15 and so on, reducing the file size to 1/6 of the original file size.

The cube file additionally stores the atoms with their atomic numbers and their corresponding atom coordinates at the beginning of the cube file.

1.1.4 Shell calculation & ESP statistics

From line 477 on, the shell of the molecule is determined. Therefore a tolerance factor of 0.12 is used, so if you run the workflow on an iso surface value of 0.001 (Politzer Convention) the shell points are the defined values where the density ρ ranges from 0.00088 to 0.00112. If the minimum amount of points (SHELL_MIN_POINTS = 50) is not reached, the tolerance is expanded to 0.3.

This function returns a Boolean array (the mask) in the same form of the grid, this functions as a template for the potential values, extracting only the needed potential values on the iso surface.

Afterwards the esp_statistics are calculated with the previously defined mask to determine the $V_{S,min}$ and $V_{S,max}$ and the total points that are located on the grid. With these values the scale is determined and the max of $V_{S,min}$ and $V_{S,max}$ is determined to create a symmetrical scale around 0.

1.1.5 Safety Measurement

From line 531 on a fallback scenario is implemented, in order to determine if the density files match the atom coordinates, with this implementation, the program will stop rendering if a mismatch is detected, hence the molecule and the ESP data mismatches. Therefore the median electron density at the supposed atomic nuclei is computed and checked for correctness. If a conversion error occurred or the wrong structure file is appended to the esp data, it is detected throughout this measurement.

1.1.6 Generating the PyMol script

From line 620 on the PyMol script is generated. That part only writes a text file, it does not talk to PyMol itself, xyzToCube.py does not even import it. The scene is built later, when the written esp.pml is started. First the stick radius is fixed at 0.10 Å in STICK_SIZE_DEFAULT and imported from here by render_esp.py and run_all.py, so the interactive scene and the rendered images cannot drift apart. Then a template string for the whole .pml file is implemented with place holders for the actual values, and from line 693 the two colouring ramps are introduced, the normal red-white-blue and the advanced rainbow ramp, ranging from red-yellow-green-cyan-blue. Line 704 puts the isosurface command together as text, isosurface surf, dens and the iso value, which is then written into the template.

1.1.7 Main program

From line 742 the main program is implemented, with all the options that are pointed out in the Pymol_ESP_Visualization/README.md

1.2 render_esp.py

The render_esp.py script is the place where the standard images are created, 3 views are standardized and always created in the same manner. One from the top looking at the pi-ring system, one from the side and one looking directly at the C-X axis pointing out the σ-hole. Within the PyMol implementation, if a molecule carries more than one halogen, the halogen with the biggest sigma hole is taken for the C-X axis view. Therefore the electrostatic potential at

the sigma holes need to be calculated, this is done by trilinear interpolation and marks a difference to the VMD implementation, where the sigma-hole potential calculation runs in a separate script.

1.2.1 Read cubes & calculate the statistics

The script starts by reading the .cube files and determine the shell with the same tolerance factor (0.12) the xyzToCube.py script also uses.

1.2.2 Determine the halogens and their corresponding values

From line 112 to line 380 the halogens are determined and the local extrema is calculated for each one. The implementation is done by searching for all halogens in the molecule that are bound to a carbon atom. From there on the local extrema for the molecule is calculated. Additionally, the electrostatic potential is calculated at the sigma holes via trilinear interpolation, trilinear interpolation is used, as there is never a shell point exactly at the C-X axis. For that a cone is implemented via a Fibonacci spiral, that goes from the C-X axis to 36.87° away from it, along every one of these 400 rays the script than marches outward from the halogen (in steps of 0.02 Bohr) and looks for the radius at which the density ρ crosses the isovalue and only there the potential is evaluated. Therefore, two values are trilinearly interpolated per ray, the crossing radius out of the density cube and the potential at that radius out of the potential cube and the maximum over all rays is where the sigma-hole sits. The angle this differs from the C-X axis is also reported (see table 3 in *ProjectElaboration.pdf*). This calculation takes a feelable increase in the computing time of the script and the reason it was left out in the image generation within the VMD Pipeline. There it is implemented in a separate script to decrease the runtime that comes with a cost of not showing the sigma hole with biggest sigma hole when there is more than one halogen in a molecule.

1.2.3 Rendering the images

From line 415 the image creation starts, for that the atomic numbers are reverse translated in their symbol again and a molecular frame is defined. The frame is taken from the geometry itself and not from the order of the atoms in the file. Via a singular value decomposition over the heavy atoms the three principal axes are determined, the axis with the smallest extent is the normal of the best fit plane of the ring system. Together with the C-X axis this spans a right-handed frame, out of which the three views are built as rotation matrices that are handed over to PyMol with set_view.

The zoom is done on the atoms and not on the iso surface, the surface object carries the extent of the whole grid box with it and the molecule would end up as a postage stamp in the middle of the image. The room the surface needs beyond the nuclei is therefore added by hand with BUFFER_ANGSTROM = 2.4, the ρ = 0.001 surface sits about 1.7 to 2.1 Å outside the outermost nuclei and 2.4 covers that with a narrow margin. The projection is set to orthoscopic, so there is no perspective distortion and the images stay comparable between molecules.

1.2.4 Creating the PyMol scene

From line 690 on the scene itself is put together. The structure file and both *.cube* files are loaded, the skeleton is shown as sticks with the stick radius that comes from --stick-size (0.1 Å by default) and the carbons are coloured grey20, so the skeleton does not compete with the surface. With *cmd.isosurface* the ρ = 0.001 surface is generated out of the density cube by a marching cubes mechanism, then a colour ramp is created with *cmd.ramp_new* on the potential cube and hung onto the surface with *surface_color*. The ramp object itself is disabled afterwards, otherwise PyMol would draw its own colour bar into the image.

Two interpolations happen here as well and it is worth keeping them apart. Marching cubes interpolates linearly along the edges of every grid cell to find where the density crosses the iso value, that gives the position of the surface vertices. The colour of each of these vertices is than the potential at that position, which PyMol trilinearly interpolates out of the ESP cube. So the geometry of the surface comes from the density and the colour from the potential.

The rest are the render settings. The transparency is 0.15 by default, PyMol counts 0 as fully opaque. Transparency_mode 2 makes the back faces of the surface visible, two_sided_lighting lights them, and specular and ambient are kept low so the surface does not glare. Cast shadows are switched off with ray_shadows 0, on a transparent surface the shadow of the skeleton reads as a dark patch of potential and competes with the colour that carries the information, the VMD pipeline switches them off as well, so both image sets are lit the same way and a comparison measures the viewer and not the lighting. Then for every background colour and every one of the three views cmd.ray renders the image at 2000 x 1600 px and cmd.png writes it out with 300 dpi.

1.2.5 Colour bar and settings file

The colour bar is deliberately not rendered into the image but written as a separate PNG from line 843 on with matplotlib, out of the same hex colours the PyMol ramp uses.

Additionally a <prefix>_settings.txt is written next to the images. It records the three input files, the grid, the iso value, $V_{S,min}$ and $V_{S,max}$ with the atoms they sit on, one line per halogen with its sigma hole and its belt, the grid spacing, the colour range and whether it was fixed or determined automatically, the ramp, the transparency, the background, the image size and the rendered views. So every image can be traced back to the parameters it was made with.

1.2.6 Main program

From line 883 the main program is implemented, with all the options that are pointed out in the *Pymol_ESP_Visualization/README.md*. If --density, --esp or --struct are not given, they are auto detected in the current folder (td.cube, tp.cube and the first .mol, .sdf or .xyz file) and the prefix of the image names is derived from the name of the structure file.

1.3 run_all.py

The run_all script enables the option for batch executions of the pipeline. The xyzToCube and the render_esp script implements the whole workflow and run_all combines the two and enables a batch execution, scanning through molecule folders and creating the standard image set and the PyMol script in one go.

1.3.1 Locate the needed files

The script starts by defining the functions for searching for all folders within the declared --root folder. Subsequently it locates the structure (*.mol, .sdf or ,xyz*), the *tp.cube* and the *td.cube* files.

1.3.2 Create .cubes and the PyMol scene

If the search for the *tp* and *td.cube* files return None, the pipeline starts with a conversion from *tp./td.xyz* files to the desired *.cube* files. It therefore uses the read_structure, read_values and write_cube function defined within the xyzToCube.py script. It furthermore continues by writing the PyMol scene with the designated function within the xyzToCube script.

1.3.3 Adapter to the render_esp.py

From line 226 on the render function follows, it does not render anything itself but is the adapter between the batch driver and the render_esp.py script. render_esp.render_all() was written for the command line and reads its options out of the Namespace object that argparse produces, therefore run_all.py builds exactly such an object again with types.SimpleNamespace and hands it over. That way there is only one render implementation in the whole project. Two of the values are not passed through but computed here, the prefix falls back to the name of the molecule folder so the images are called brombenzol_pi.png and not molecule_pi.png, and the outdir is assembled to the full path, because run_all.py never changes the working directory and works with absolute paths throughout. That is a difference to the VMD side, there run_allVMD.py does an os.chdir into the molecule folder before rendering, because render_espVMD.py works in the current directory, where esp.tcl and the cube files sit next to each other.

1.3.4 Creating the summary.csv for the batch execution

From line 251 on, a function is created to define an exported summary.csv export and to subsequently fill the summary with the important information of the batch execution.

1.3.5 Main program

From line 335 to the end of the script the main function and all its parameters are defined. Find out more in the designated *README.md* file. Here the batch execution is rolled out in its full expense.

2. VMD pipeline

With the implementation of a second pipeline in the VMD visualization tool, a control mechanism was implemented, validating the results of the first implementation. Furthermore, a second set of images is produced, that changes slightly and has its advantages. The atoms and its sticks are way better observable than in the PyMol implementation, but the sigma-hole in at the corresponding halogen atom is not that clear anymore (See Figure 1 and 2). The general implementation is the same, but changes quite a few times within these tools, which is pointed out in this document.

2.1 xyzToCubeToVMDVis.py

This script has the same purpose than the xyzToCube.py script in the PyMol implementation. The data of the density and potential comes in huge *td./tp.xyz* files and needs conversion to Gaussian *.cube* files. The script also creates the interactive scene of the molecule within VMD, with the help of the esp_template.tcl script. The difference here is that the VMD template lives in a separate file, while the PyMol template is just a string in the corresponding python script.

2.1.1 Reading turbomole and structure files, converting units

Until line 261 the script is the same as the PyMol implementation, the structure and *.xyz* files get read, the units are correctly converted, if needed and the important values of the density and potential files are extracted.

2.1.2 Defining the shell and compiling the statistics

From line 261 until 300 the shell at the isovalue +-12% is extracted of the density and potential files. With these potential values the $V_{S,Min}$ and $V_{S,Max}$ is computed, in order to define the scale of the created VMD scene. Within this implementation the $V_{S,Min}$ and $V_{S,Max}$ are additionally added into the header line of the cube files, enabling the --tcl-only option, that only rewrites the scene and not calculate the *.cube*s again. This is a feature only implemented within the VMD implementation.

2.1.3 Reading & Writing the cube files

From line 300 to 450, the functions to read and write the cube files are implemented. As stated the *tp.cube* files are implemented with writing the range in its header, to store the values for the tcl scene creation, enabling the --tcl-only option. Again, with the *.part* suffix, until the .cube file is fully written (see 1.1.3.).

2.1.4 Defining the view

From line 452 on the views get defined. The frame itself is the same calculation as in the PyMol pipeline, see 1.2.3, that is done by s purpose: a singular value decomposition over the heavy atoms delivers the three principal axes, the axis with the smallest extent becomes the normal of the best fit plane of the ring system and the reference axis is not an inertial axis at all but the bond itself, C → X, kept twice, once as the true bond axis and once projected into the plane. The comment in line 435 states why it was taken over line by line. What is different is where it happens. In PyMol the orientation is computed at render time inside render_esp.py, here it is computed at conversion time and baked into the generated *esp.tcl*, the scene already carries its views before VMD is started at all.

From line 508 on the three views are built out of these vectors as rotation matrices. The pi view looks along the normal with the C-X axis pointing down, the edge view looks along the cross product of normal and axis, so that the C-X axis lies horizontal in the image and the sigma view looks in from outside along the true, unprojected bond axis. Two things are added here that the PyMol side does not have. The tilt of the C-X axis against the best fit plane is calculated and handed back, so that a molecule in which the sigma view and the pi view diverge is on record instead of silently producing a misleading picture. Secondly if no atoms are available at all, a fallback set of world axes reproduces exactly the fixed rotations the earlier template used.

From line 534 the matrices are formatted as 4x4 Tcl lists and are than written into the generated *esp.tcl* as the constants ROT_PI, ROT_EDGE and ROT_SIGMA. Here the second difference to PyMol shows up and it is a single line: view_matrix hands back the camera basis as rows, while the PyMol version transposes it before returning. Both describe the same camera, cmd.set_view simply expects the basis vectors as columns and molinfo rotate_matrix expects them as rows. Writing the matrices out as constants is also what enables the usage of esp_view pi/sigma/edge within the TK console of VMD and marks a difference to the PyMol implementation, where these options are not available. The zoom is not part of this and is handled differently on both sides, PyMol adds a fixed buffer of 2.4 Å around the atoms while VMD scales the scene over --scale and --fill, see section 2.1.6.

2.1.5 Safety Measurement

The same fallback scenario as in 1.1.5 is implemented within the VMD pipeline as well, the script checks for the electron density median at the atomic coordinates and cross-checks the plausibility.

2.1.6 Generating the VMD script -- esp_template.tcl

From line 665 to 795 the VMD scene is created, therefore, unlike in the PyMol implementation, where a string was created within the python script with placeholders that are filled in, a separate *esp_template.tcl* script is used. For that the colour ramps need to be defined and correctly passed on and the all the placeholders are filled in with the determined values, creating a fully functional *esp.tcl* script throughout the execution of the program.

2.1.7 Main program

From line 796 on the main program is defined, implementing all the options, mentioned in the *Readme.md*

2.2 render_espVMD.py

The render_espVMD.py script is again responsible for creating the standard image set for the VMD pipeline, but it follows a different implementation method: it does not read the cube data at all. Instead, it starts VMD as a subprocess, which sources the esp.tcl scene created earlier, that scene brings everything with it. The isovalue, the colour scale, the three rotation matrices ROT_PI, ROT_EDGE and ROT_SIGMA and the esp_view command that selects between them. The python side only hands over the names of the views it wants and picks up two numbers for its own output, the isovalue and the colour range out of *esp.tcl* and the $V_{S,min}$, $V_{S,max}$, the colour range, the iso value and the amount of shell points out of the first header line of *tp.cube*, where xyzToCubeToVMDVis.py stamped them during the conversion. Therefore the script does not analyse the volumetric data at all, all of that has already been calculated in the xyzToCubeToVMDVis.py script.

This results that the script cannot be run standalone, the finished *esp.tcl* file needs to be done and if the user wants to change the isovalue, the esp range or other parameters, the *esp.tcl* file needs to be recreated.

2.2.1 Locate & run vmd

The script starts with locating the vmd application on the computer and subsequently initializes a VMD session, if the program isn't able to locate vmd, the path can be manually added with the --vmd option.

After locating vmd, the script does not hand over the *esp.tcl* scene directly. VMD takes no variables from the command line, its -e option only runs a script, therefore the options are written as tcl assignments into a throwaway file _render_opts.tcl in the molecule folder: the window size, the background, the target folder and prefix, which of the three views to render, which scene file to use and whether this pass should render opaque or as a window capture. The last line of that file sources the second tcl script of the project, render_esp.tcl, which lives next to the python scripts and is the same for every molecule.

VMD is than started with vmd -e _render_opts.tcl as a subprocess. It executes the option file, so the variables are set, and then loads render_esp.tcl, which checks every variable with info exists and falls back to a default if it is missing. That way the tcl script can also be started by hand from a molecule folder, without python, and still produces the same images. It sources the esp.tcl scene, so that the molecule, the isosurface, the colour scale and the three rotation matrices are available, sets the display and render options and then walks through the requested views, calling esp_view and render for each of them, both wrapped in a catch so that one failing view does not take the others with it. At the end it quits VMD, which is what lets the python side wait for the process instead of guessing when it is done. The complete VMD output is appended to *images/_vmd.log*, and the option file is deleted afterwards, so nothing of it stays in the molecule folder.

2.2.2 Reading the scene back

From line 139 on the script picks up the few numbers it needs for its own output. It does not parse the scene, it greps it: two regular expressions pull set ISO and set RANGE out of the *esp.tcl* and a third one reads the statistics stamp out of the first header line of *tp.cube*. Therefore the whole analysis crosses over into this script as text, and none of the 15.8 million grid points must be touched for a second time.

2.2.3 Converting the images

From line 159 on the images are converted. VMDs Tachyon writes TGA files, which no document format takes, so every rendered view is opened with PIL and saved again as a PNG and the TGA is deleted afterwards unless the run was started with --keep-tga.

2.2.4 Colour bar and settings file

From line 184 the colour bar is generated with matplotlib, as a separate PNG and out of the same hex colours the PyMol pipeline uses, so that the two bars are comparable and the images themselves stay free of text. VMD has no legend object at all, so this is not a convenience but the only way to ship a scale with a VMD image.

From line 220 the settings file is written. It records all necessary information about the image creation run and additionally records for each of the three views by which pass it was produced, which matters because not every view is necessarily rendered at full quality.

2.2.5 Rendering in passes

From line 269 the actual driver sits and it is built around the fact that VMDs ray tracer is not reliable on this scene (Unlike PyMol, where the images are reliably produced). Three passes are defined: the first renders with transparency and Tachyon, the second repeats only the views that are still missing as a window capture, which keeps the transparency but is less fine and the third renders the remainder opaque. After every pass the script checks which TGA files now exist, and it takes a time stamp before each run, so that a leftover file from an earlier, crashed pass cannot be counted as a success. Whatever a pass produced is kept and every view remembers which pass produced it. Each background colour goes through this logic on its own, because a Tachyon run that survives on white says nothing about whether it survives on black. Only after that the images are converted, the colour bar is drawn and the settings file is written.

2.2.6 Main program

From line 396 the main program is implemented, with all the options that are pointed out in the VMD_ESP_Visualization/README.md.

2.3 run_allVMD.py

The run_allVMD script does the same job as run_all.py on the PyMol side, it scans through the molecule folders below --root, converts what still needs converting, writes the esp.tcl scene and renders the standard image set for every molecule in one go and writes a *summary.csv* at the end. The differences to the PyMol implementation follow from where the work sits in each pipeline.

The first one is the working directory. render_espVMD.py works in the current folder, because *esp.tcl* and the two cube files must lie next to each other, therefore run_allVMD.py does an os.chdir into every molecule folder before rendering and changes back afterwards, while run_all.py never changes the directory and assembles absolute paths instead. The second one is that no adapter is needed here, render_espVMD.render_all() takes normal keyword arguments and the batch driver calls it directly, where the PyMol side first has to rebuild the argparse object with types.SimpleNamespace (1.3.3).

The third and biggest difference is which script the parameters go to. The isovalue, the colour range, the opacity, the zoom and the stick size are all baked into the scene at conversion time, therefore run_allVMD.py hands them to xyzToCubeToVMDVis.py, while on the PyMol side the same values are handed to the renderer, because the PyMol scene does not carry them. And at the end, where run_all.py offers --two-pass and simply renders everything a second time with the largest range found, run_allVMD.py only prints the exact command line to do so, with the fixed --esp-range already filled in. Which scale is the informative one is a decision that belongs to the user and not to the script.

Find out more in the designated *README.md* files.

3. Calculation differences in the pipelines

	PyMol pipeline	**VMD pipeline**
$V_{S,min}$ / $V_{S,max}$	grid points in the ρ = iso ± 12 % band	the same rule, the same numbers
σ-hole	located per halogen inside the render pass	tools/SigmaHoleCalc.py, run separately
Halogen belt	reported with every figure	reported by the same separate tool
Which atom carries the extremum	reported (VS_max_on, VS_min_on)	not determined
Several halogens in the same molecule	all evaluated and ranked, strongest first	not evaluated, view takes the first found halogen
Units in the summary	a.u., kcal/(mol·e), kJ/(mol·e)	a.u., kJ/(mol·e)

Table 1: What each pipeline reports as part of a normal run. The first row is not a difference: both take the surface extrema from the same band by the same rule, which is why the values in ProjectElaboration.pdf §3.6 agree exactly.